BACKEND ENGINEERING · TECHNICAL DOCUMENTATION

Rate Limiting Middleware

A pluggable FastAPI + Redis throttling layer implementing Fixed Window, Sliding Window Log and Token Bucket.

DOCUMENT	**Technical Design & Implementation Reference**
STACK	FastAPI (ASGI) · Redis · Python 3.9+
ALGORITHMS	Fixed Window Counter · Sliding Window Log · Token Bucket
COMPONENTS	`fixed_window_middleware.py` , `sliding_window_middleware.py` , `Rate_limiter_middleware_TokenBucket.py` and their test harnesses
AUDIENCE	Backend engineers, reviewers, technical interviewers
VERSION	1.0

> This document describes the architecture, algorithmic behaviour, Redis data model, operational procedures and known limitations of the rate limiting middleware. Each algorithm is presented with a design diagram, a step-by-step execution trace, complexity analysis and an annotated reading of the implementation.

Contents

FIGURES

1. Project Overview

Rate limiting is the control plane that stands between a public endpoint and unbounded resource consumption.

1.1 The Problem

Any publicly reachable HTTP endpoint is, by default, an open invitation to consume unlimited server resources. Without a throttling layer:

- A single misbehaving client can saturate CPU, connection pools, or third-party API quotas.
- Credential-stuffing and brute-force attacks execute at full network speed against login endpoints.
- Cost-per-request infrastructure — LLM APIs, SMS gateways, payment processors — becomes financially unbounded.
- One noisy tenant degrades latency for every other tenant: the classic *noisy neighbour* problem.

1.2 The Solution

This project implements rate limiting as **ASGI middleware**: a layer that intercepts every request *before* it reaches a route handler, consults shared state in Redis, and either forwards the request or short-circuits it with `429 Too Many Requests`.

Three algorithms are implemented behind a common interface, so the enforcement strategy becomes a deployment decision rather than an architectural one.

ALGORITHM	ENFORCEMENT CHARACTER	TYPICAL USE
Fixed Window	Coarse, cheapest	Hourly or daily quotas where smoothness is irrelevant
Sliding Window Log	Exact, memory-heavy	Hard security limits — login attempts, OTP requests
Token Bucket	Smooth average, burst-tolerant	General-purpose public API traffic

1.3 Why Redis

In a distributed system, application servers are stateless and horizontally scaled. If each instance tracked counters in local memory, a client hitting four instances behind a load balancer would effectively receive four times the intended quota. Rate limit state must therefore live outside the process.

REQUIREMENT	HOW REDIS SATISFIES IT
Shared state	Every application instance reads and writes the same counters
Low latency	In-memory storage; sub-millisecond command execution
Atomicity	Single-threaded command loop; `INCR`, `ZADD` and Lua scripts execute indivisibly
Automatic cleanup	Native key expiration via `EXPIRE` / TTL
Fit-for-purpose structures	Hashes for counters and buckets; Sorted Sets for timestamp logs

> **Design principle.** The limiter never trusts process-local memory. Every decision is derived from state that is visible to, and mutable by, every node in the cluster.

2. System Architecture

Client → FastAPI → Middleware → Redis → Decision → Response.

2.1 High-Level Flow

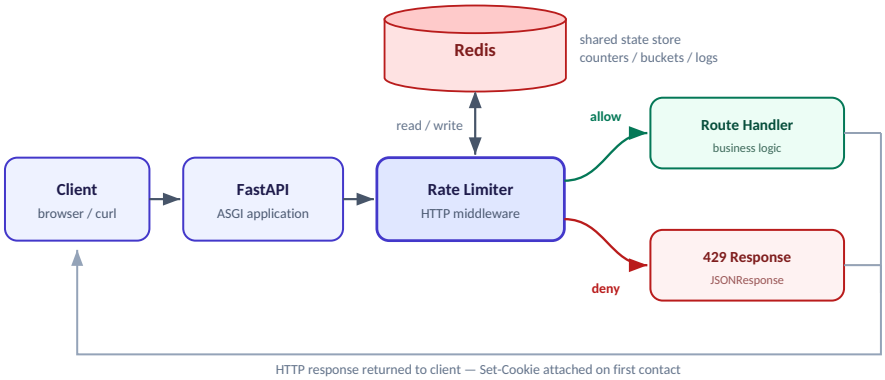

Figure 1. End-to-end request architecture. Redis is consulted on every request, before any business logic executes.

2.2 Component Responsibilities

COMPONENT	RESPONSIBILITY
Client	Issues requests and stores the `user_id` cookie returned on first contact
FastAPI app	ASGI application; owns the middleware stack and the route table
Identity resolution	Reads `user_id` from cookies; mints a UUID4 and raises an `is_new` flag when absent
Limiter core	Pure decision function: given an identity, returns *allow* or *deny*. No I/O beyond Redis
Redis	Single source of truth for per-user state, shared across all application instances
Route handler	Business logic; executes only for permitted requests

2.3 The Factory Pattern

Every algorithm exposes a factory function rather than a bare middleware coroutine:

```
def rate_limiter_middle_ware(app, capacity, refill_rate):
    @app.middleware("http")
    async def Rate_limiter_middleware(request, call_next):
        ...
```

The factory **closes over configuration** — `capacity`, `refill_rate`, `window`, `limit` — so the middleware requires no module-level globals and no class instantiation at import time. Registration collapses to a single line in the application entrypoint:

```
app = FastAPI()
rate_limiter_middle_ware(app, capacity=10, refill_rate=1)
```

This is what makes the design extensible. A new algorithm only has to expose a function of the same shape; nothing about the application entrypoint changes, and multiple configurations can coexist in the same process.

2.4 Request Lifecycle

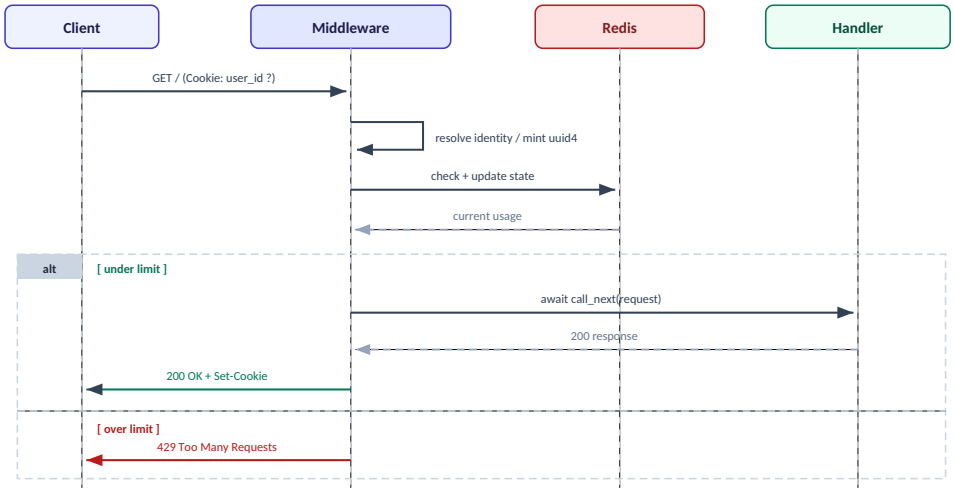

Figure 2. Request lifecycle. The handler is never reached when the limit is exceeded — rejection costs one Redis round trip.

3. Algorithm Breakdown

Three strategies, one interface. Each trades accuracy against memory and burst tolerance differently.

3.1 Fixed Window Counter

fixed_window_middleware.py

EXPLANATION

Time is divided into fixed-length windows — ten seconds, one hour, one day. Each user holds a counter and a window start timestamp. Every request increments the counter; once it reaches `max_count` within the current window, further requests are rejected. When the window elapses, the counter resets to one and a new window opens.

WHY USE IT

It is the simplest correct approach and the cheapest to run: two integers per user, O(1) operations, trivially debuggable. It is the right choice for coarse quotas such as *"1,000 API calls per hour"*, where perfect smoothness carries no business value.

PROS & CONS

ADVANTAGES	TRADE-OFFS
Minimal memory — two hash fields per user	**Boundary burst:** up to 2× the limit across a window edge
O(1) time; no scanning, no sorted structures	Traffic clusters at the start of each window
Easy to explain, easy to reason about in review	The window is per-user, not aligned to a global clock
Maps directly onto Redis `INCR` + `EXPIRE`	Coarse-grained fairness between clients

> **The boundary burst problem.** With a limit of 10 requests per 10 seconds, a client can send 10 requests at t = 9.9s and 10 more at t = 10.1s. That is 20 requests inside 200 milliseconds, every one of them legally accepted.

COMPLEXITY

DIMENSION	COST	REASON
Time per request	`O(1)`	Two hash field reads and one write
Space per user	`O(1)`	A hash with `counter` and `start_point`
Total space	`O(U)`	*U* = number of tracked identities

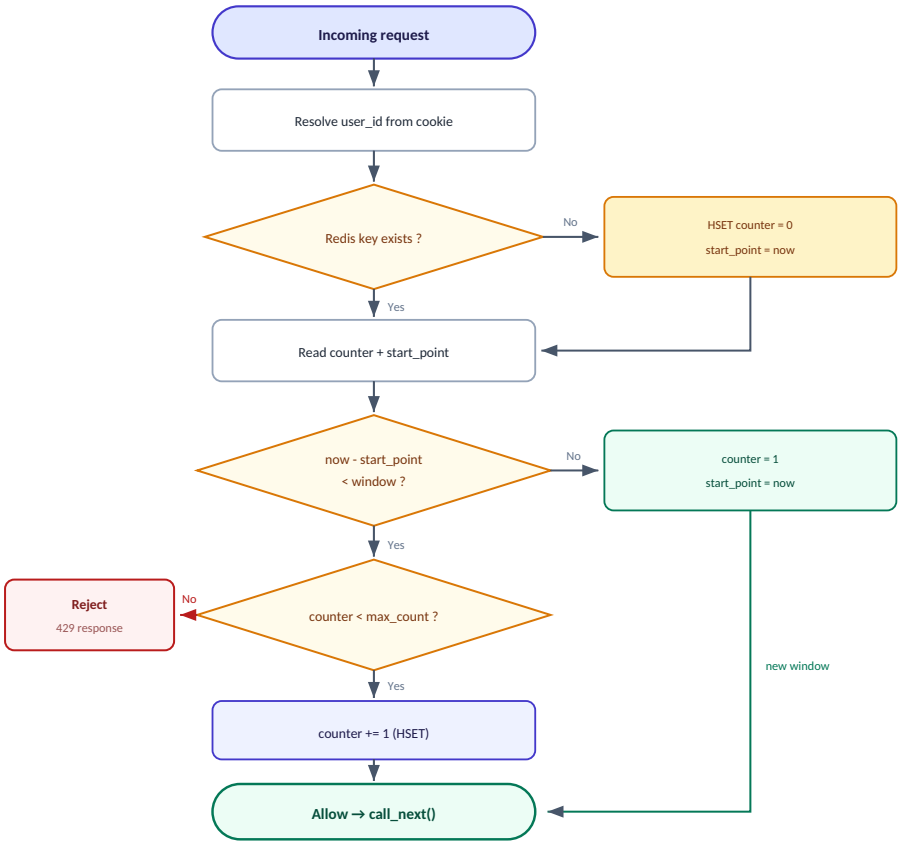

Figure 3. Fixed Window decision flow. Two exits only: increment-and-allow, or reject.

STEP-BY-STEP EXECUTION

1. Read `user_id` from the request cookies; mint a UUID4 and set `is_new_user = True` when absent.
2. `EXISTS user_id` — if the hash is missing, initialise `counter = 0` and `start_point = now`.
3. `HGET counter` and `HGET start_point`.
4. Compute the elapsed time: `now - start_point`.
5. **Window expired** → reset `counter = 1`, `start_point = now`, allow.
6. **Inside the window and `counter < max_count`** → increment, allow.
7. **Inside the window and at the ceiling** → reject with 429.
8. On allow, `await call_back(request)` and attach an `httponly` cookie for new users.

CODE EXPLANATION

```
if (now - start_point) < fixed_window:
    if counter < max_count:
        counter += 1
        r.hset(name=user_id, key="counter", value=counter)
        return True
    else:
        return False
else:
    r.hset(name=user_id, mapping={"counter": 1, "start_point": time.time()})
    return True
```

The window is **rolling per user, not globally aligned**. `start_point` is stamped when that user's first request of a window arrives, not on a wall-clock boundary, so every client has its own window phase. This is a deliberate simplification: it removes the need for a shared clock at the cost of making quotas harder to reason about in aggregate.

Note that the reset branch writes `counter = 1`, not `0` — the request that triggered the reset is itself counted. Off-by-one errors here are a common source of silent quota drift.

Implementation note. The module currently registers `@app.middleware("http")` at module scope against a module-level `app` object *and* exposes the `RateLimiter_FixedWindow_MiddleWare(app)` factory containing duplicated helpers. Only the factory is used by `fixed_window_test.py`; the module-level application and its copies are dead code and should be deleted.

3.2 Sliding Window Log

sliding_window_middleware.py

EXPLANATION

Instead of a counter, every accepted request is recorded as an entry in a Redis **Sorted Set**, scored by its Unix timestamp. On each new request the middleware deletes all entries older than `now - window`, counts what remains, and admits the request only if that count is below the limit.

The window is therefore *continuous*: it always covers exactly the last N seconds relative to the current instant, rather than a pre-defined block of time.

WHY USE IT

This is the most accurate of the three algorithms. It structurally eliminates the boundary burst — the limit holds over *every possible* N-second interval, not merely over aligned windows. Use it wherever the quota is a hard contract: login throttling, OTP issuance, paid API tiers, anti-abuse enforcement.

PROS & CONS

ADVANTAGES	TRADE-OFFS
Exact — boundary bursts are impossible	Memory scales with the limit: O(limit) per user
Enforces the limit over any arbitrary interval	Sorted-set writes cost O(log N), not O(1)
Retains a full request history, useful for debugging	Higher Redis memory pressure at scale
Self-cleaning through `ZREMRANGEBYSCORE`	Three round trips unless collapsed into a Lua script

COMPLEXITY

OPERATION	COST	NOTES
`ZREMRANGEBYSCORE`	`O(log N + M)`	*M* = entries actually removed
`ZCARD`	`O(1)`	Cardinality is maintained internally
`ZADD`	`O(log N)`	Skip-list insertion
Space per user	`O(limit)`	The set never exceeds the limit after trimming
Total space	`O(U × limit)`	The dominant scaling concern

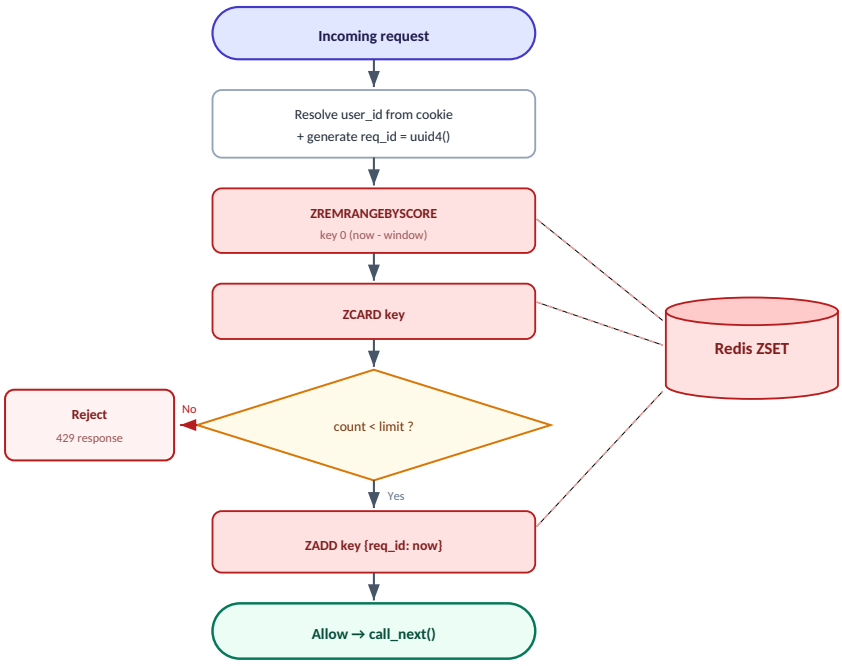

Figure 4. Sliding Window Log decision flow. Red nodes are Redis commands; the trim always precedes the count.

STEP-BY-STEP EXECUTION

1. **Extract** `user_id` from cookies; generate a UUID4 for first-time clients.

2. **Generate** `req_id` — a fresh UUID4 that will act as the unique sorted-set member.
3. **Purge expired entries:** `ZREMRANGEBYSCORE user_id 0 (now - window)`.
4. **Count survivors:** `ZCARD user_id`.
5. **Compare** the count against the configured limit.
6. **Accept** → `ZADD user_id {req_id: now}` and forward to the handler, or **reject** with 429.

SORTED SET STATE

The table below shows the ZSET for one user at t = 12.0s with `window = 10` and `limit = 5`. Entries scored below 2.0 fall outside the window and are removed before counting.

MEMBER (REQ_ID)	SCORE (TIMESTAMP)	STATUS AT T = 12.0S
~~`req-1`~~	1.4	Older than 2.0 — trimmed by `ZREMRANGEBYSCORE`
`req-2`	4.2	Inside the window
`req-3`	7.8	Inside the window
`req-4`	11.1	Inside the window
ZCARD after trim = 3 → 3 < 5 → **ALLOW**, then `ZADD req-5 → 12.0`		

CODE EXPLANATION

```
def sliding_window(user_id, req_id, window=10, limit=5):
    now = time.time()
    r.zremrangebyscore(user_id, 0, now - window)
    number_of_requests = r.zcard(user_id)
    if number_of_requests < limit:
        r.zadd(user_id, {req_id: now})
        return True
    return False
```

Why a UUID member rather than the timestamp? Sorted set members must be unique. Two requests landing in the same floating-point microsecond would collide, silently overwriting one another and undercounting usage. A UUID guarantees every accepted request occupies its own slot.

Trim before counting, unconditionally. The `ZREMRANGEBYSCORE` call is what makes the window slide. Skipping it when the count is already at the limit would permanently lock the user out, since the set would never shrink.

Rejected requests are not logged. The denial path never calls `ZADD`, so a client hammering a closed door does not extend its own penalty. This is the correct default for most APIs; invert it only if punitive backoff is an explicit product requirement.

> **Implementation note.** The `window` and `limit` parameters are currently fixed in the inner function's signature rather than threaded through the `SlidingWindow_MiddleWare(app)` factory. Lifting them into the factory would bring this module in line with the token bucket implementation and make the limit configurable per deployment.

3.3 Token Bucket

Rate_limiter_middleware_TokenBucket.py

EXPLANATION

Each user owns a bucket holding up to `capacity` tokens. Tokens refill continuously at `refill_rate` tokens per second. Every request consumes one token; when the bucket is empty, the request is rejected.

Rather than running a background refill job, tokens are computed **lazily** at request time:

```
available = min(capacity, stored_tokens + elapsed_seconds * refill_rate)
```

This is the key insight of the algorithm: the refill is a pure function of elapsed time, so no timer, cron job, or background task is required anywhere in the system.

WHY USE IT

Token Bucket is the industry default — AWS API Gateway, Stripe and NGINX's `limit_req` all use variants of it — because it cleanly separates **average rate** from **burst tolerance**. A client that has been idle accumulates tokens and may spend them in a burst, which is exactly how legitimate clients behave, while the long-run average stays capped at `refill_rate`.

PROS & CONS

ADVANTAGES	TRADE-OFFS
Permits controlled bursts up to `capacity`	Two tunable parameters instead of one
Smooth long-run average rate	Requires floating-point arithmetic; precision matters
O(1) time and space per user	Read-modify-write is racy without a Lua script
No background refill process needed	Marginally less intuitive to explain to stakeholders

COMPLEXITY

DIMENSION	COST	REASON
Time per request	`O(1)`	One `HGETALL`, one `HSET`
Space per user	`O(1)`	A hash with `tokens` and `last_time`
Total space	`O(U)`	Independent of traffic volume

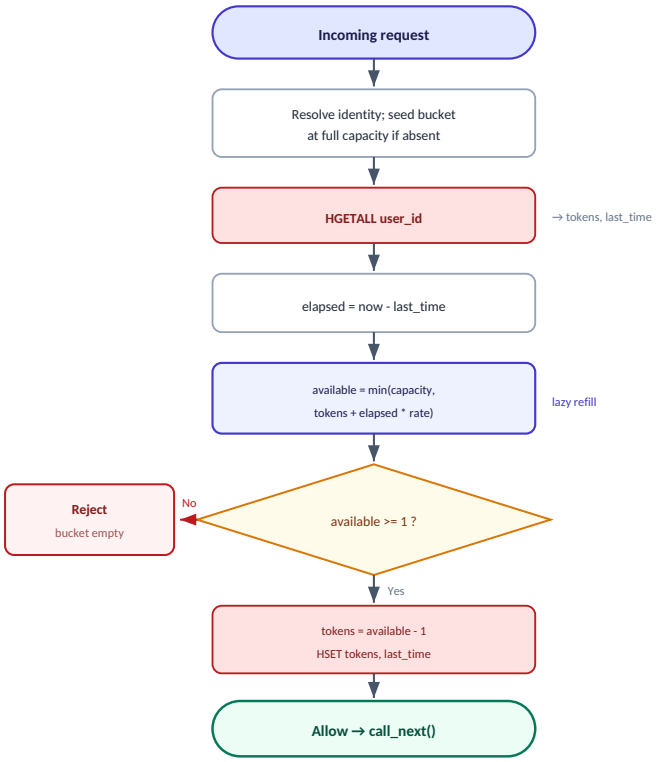

Figure 5. Token Bucket decision flow. Refill happens arithmetically at read time — no scheduler involved.

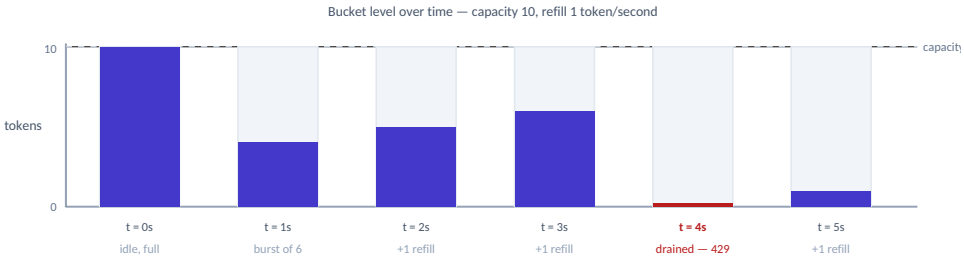

Figure 6. An idle client banks tokens up to the ceiling; a burst drains them; refill is linear and continuous.

STEP-BY-STEP EXECUTION

1. **Resolve identity** — read the cookie, or mint a UUID4 and seed the bucket at full capacity.
2. **Defensive re-seed** — if the Redis key has expired or been evicted, re-initialise it before reading.
3. `HGETALL` the bucket hash: `tokens` and `last_time`.
4. **Compute elapsed time** since the last accepted request.
5. **Refill:** `min(capacity, tokens + elapsed * refill_rate)` — the `min` enforces the ceiling.
6. **If at least one token is available:** decrement, write back both fields, allow.
7. **Otherwise:** return 429 without mutating Redis.

CODE EXPLANATION

```
now = time.time()
data = r.hgetall(name=self.user_id)
last_time = float(data['last_time'])
tokens = int(data['tokens'])
difference = now - last_time
amount = min(self.capacity, int(difference * self.refill_rate) + tokens)
```

The `min(capacity, ...)` clamp is the heart of the algorithm. Without it, an idle client would accumulate tokens without bound and could later flood the service with a single enormous burst — the ceiling is what converts "unused quota" into "burst allowance" rather than "stored ammunition".

Precision caveat. `int(difference * self.refill_rate)` truncates fractional tokens, and because `last_time` is reset to `now` on every accepted request, that truncated remainder is discarded rather than carried forward. With `refill_rate = 1` and steady sub-second traffic, `int(0.4)` evaluates to `0` on every call and the bucket may never refill at all. Store `tokens` as a float and truncate only at the comparison:

```python
# tokens stored as float; truncation deferred to the comparison
amount = min(self.capacity, tokens + difference * self.refill_rate)
if amount >= 1:
    amount -= 1
```

3.4 Comparison Matrix

CRITERION	FIXED WINDOW	SLIDING WINDOW LOG	TOKEN BUCKET
Accuracy	Low — 2× boundary burst	Exact	High
Burst handling	Uncontrolled at window edges	Blocked entirely	Controlled and configurable
Time complexity	`O(1)`	`O(log N + M)`	`O(1)`
Space per user	`O(1)`	`O(limit)`	`O(1)`
Redis structure	Hash	Sorted Set	Hash
Parameters	`window` , `max_count`	`window` , `limit`	`capacity` , `refill_rate`
Best suited to	Coarse hourly or daily quotas	Hard security limits	General-purpose public APIs

Selection guidance. Start with Token Bucket for public API traffic; it is forgiving to well-behaved clients and strict on sustained abuse. Escalate to Sliding Window Log only on endpoints where an over-admission is a security incident rather than a performance annoyance.

4. Redis Data Design

The choice of data structure is the choice of algorithm; each limiter is defined as much by its key layout as by its code.

4.1 Why a Sorted Set for the Sliding Window

A Sorted Set (`ZSET`) is the only native Redis structure that supports **range deletion by score in logarithmic time** — precisely the operation a sliding window needs on every request.

ALTERNATIVE	WHY IT FAILS
List	Trimming by timestamp requires an O(N) scan from the head
String counter	Carries no per-request timestamps, so the window cannot slide
Hash	No ordering and no range queries by value
Set	Unordered, with no score dimension to filter on

With a ZSET, `ZREMRANGEBYSCORE key 0 (now - window)` evicts the entire expired prefix in a single atomic command.

4.2 Key and Value Structures

SLIDING WINDOW — SORTED SET

```
KEY:    <user_id>                    e.g. "3f2b8c1a-...-9d4e"
TYPE:   ZSET
MEMBER: <request_id>                 UUID4 - guarantees uniqueness
SCORE:  <unix_timestamp>             float - the sliding dimension

ZADD  3f2b8c1a-...  1721822400.512  "a91f-..."
ZCARD 3f2b8c1a-...                            -> 4
```

TOKEN BUCKET — HASH

```
KEY:  <user_id>
TYPE: HASH
  tokens     : int    - tokens currently held
  last_time  : float  - timestamp of the last accepted request
```

FIXED WINDOW — HASH

```
KEY:  <user_id>
TYPE: HASH
  counter      : int    - requests served in the current window
  start_point  : float  - timestamp at which the window opened
```

4.3 TTL and Cleanup Logic

Cleanup currently operates at two of the three levels it needs to.

LEVEL	MECHANISM	STATUS
Intra-key	`ZREMRANGEBYSCORE` trims expired entries on every sliding-window request	**Implemented**
Intra-key	Fixed window resets `counter` when the window elapses	**Implemented**
Whole-key	Redis `EXPIRE` on idle identities	**Missing**

> **This is the most consequential gap in the current design.** Because `user_id` is a per-browser UUID and no key ever expires, Redis memory grows linearly and permanently with the number of unique visitors. Under sustained traffic this ends in eviction storms or an out-of-memory failure.

Every algorithm should set a TTL alongside its state write:

```
# Sliding window
r.zadd(user_id, {req_id: now})
r.expire(user_id, window * 2)                        # idle users evaporate

# Token bucket - long enough to refill from empty to full
r.hset(user_id, mapping={...})
r.expire(user_id, int(capacity / refill_rate) + 60)

# Fixed window
r.hset(user_id, mapping={...})
r.expire(user_id, fixed_window * 2)
```

The TTL must always exceed the window. If it does not, state is dropped mid-window and the user silently receives a fresh quota — a subtle bypass that is difficult to detect in production. Refresh the TTL on every write so that active users are never evicted mid-flight.

A namespaced key prefix is also recommended, so limiter keys can be inspected, scoped or flushed independently of any other data in the same Redis instance:

```
ratelimit:sliding:<user_id>
ratelimit:bucket:<user_id>
ratelimit:fixed:<user_id>
```

5. Running the Project

5.1 Prerequisites

Python	3.9 or newer
Redis	6 or newer — local install or Docker
Packages	`fastapi` , `uvicorn` , `redis`

5.2 Step 1 — Install Dependencies

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

pip install fastapi uvicorn redis
```

5.3 Step 2 — Start Redis

```
# Docker (recommended)
docker run -d --name redis-limiter -p 6379:6379 redis:7-alpine

# Or a local installation
redis-server
```

Verify connectivity before starting the application:

```
redis-cli ping        # -> PONG
```

5.4 Step 3 — Run the API

Each algorithm has its own entrypoint. Start whichever one you want to exercise:

```
# Sliding Window    ->  GET /
uvicorn sliding_window_test:app --reload --port 8000

# Token Bucket      ->  GET /
uvicorn RateLimiter_TokenBucket_Test:app --reload --port 8000

# Fixed Window      ->  GET /login
uvicorn fixed_window_test:app --reload --port 8000
```

Interactive OpenAPI documentation is served at `http://localhost:8000/docs` .

5.5 Configuration Reference

ALGORITHM	PARAMETERS	DEFAULT	CONFIGURED IN
Sliding Window	`window` , `limit`	10s, 5 requests	`sliding_window()` signature
Token Bucket	`capacity` , `refill_rate`	10 tokens, 1/s	`RateLimiter_TokenBucket_Test.py`
Fixed Window	`fixed_window` , `max_count`	10s, 10 requests	`Fixed_window()` call site

6. Testing the Rate Limiter

6.1 A Single Request

```
curl -i -c cookies.txt http://localhost:8000/
```

> **The cookie jar matters.** `-c cookies.txt` persists the `user_id` cookie. Without it every request looks like a brand-new client, each gets its own quota, and the limiter never appears to fire.

Expected first response:

```
HTTP/1.1 200 OK
set-cookie: user_id=3f2b8c1a-...; HttpOnly
content-type: application/json

{"message": "hello from home page.."}
```

6.2 Triggering the Limit

```
for i in $(seq 1 10); do
  curl -s -o /dev/null -w "req $i -> %{http_code}\n" \
    -b cookies.txt -c cookies.txt http://localhost:8000/
done
```

Expected output with `limit = 5` and `window = 10s`:

```
req 1  -> 200
req 2  -> 200
req 3  -> 200
req 4  -> 200
req 5  -> 200
req 6  -> 429
req 7  -> 429
req 8  -> 429
req 9  -> 429
req 10 -> 429
```

6.3 Verifying Recovery

```
sleep 11 && curl -i -b cookies.txt http://localhost:8000/
# -> 200 OK: the window has slid past the earlier entries
```

6.4 Inspecting Redis State

```
redis-cli

> KEYS *
> ZRANGE <user_id> 0 -1 WITHSCORES     # sliding window log
> HGETALL <user_id>                    # token bucket / fixed window
> TTL <user_id>                        # -1 means no expiry - see section 4.3
```

6.5 Postman

1. Send `GET http://localhost:8000/` — Postman retains cookies per domain automatically.
2. Re-send repeatedly, or use the Collection Runner with 10 iterations and no delay.
3. Observe the status code flipping from `200` to `429` at the configured threshold.

6.6 Test Matrix

SCENARIO	EXPECTED RESULT
First-ever request	`200` with `Set-Cookie: user_id`
Requests within the limit	`200`
Request exceeding the limit	`429`
Request after the window elapses	`200`
Request sent without a cookie	`200` — treated as a new identity
Two independent cookie jars	Independent quotas, no interference
Redis stopped mid-test	`500` — see limitation 10

7. Known Limitations

An honest inventory of what stands between this implementation and production traffic.

#	ISSUE	IMPACT	RESOLUTION
1	**Fixed Window returns its rejection with the default `200` status.** `JSONResponse(content="Rate limit exceeded...")` omits `status_code`.	Clients receive a success status on a rejection and will never back off. Silently breaks every well-behaved consumer.	Add `status_code=429`
2	**Identity is a client-controlled cookie.** Clearing cookies grants a fresh quota.	Trivially bypassed by any attacker; the limiter constrains honest clients only.	Key on the authenticated user ID or API key, with client IP as fallback
3	**No TTL on any Redis key.**	Unbounded memory growth proportional to unique visitors.	Set and refresh `EXPIRE` on every write — section 4.3
4	**Read-modify-write is not atomic.** Concurrent requests interleave between the read and the write.	Under concurrency the effective limit is overshot; the error grows with parallelism.	Collapse the check into a Lua script (`EVAL`) or a `MULTI` / `EXEC` pipeline
5	**Synchronous `redis-py` client inside `async` middleware.**	Every Redis call blocks the event loop, capping throughput far below the framework's capability.	Switch to `redis.asyncio` and await the calls
6	**Token bucket truncates fractional tokens.**	Buckets may never refill under sustained sub-second traffic.	Store `tokens` as a float — section 3.3
7	**Sliding window limits are hardcoded** in the inner function rather than parameterised through the factory.	Cannot be tuned per deployment or per environment.	Lift `window` and `limit` into the factory signature
8	**Sliding window cookie is not `httponly`.**	Reachable from JavaScript; inconsistent with the other two implementations.	Add `httponly=True` , plus `secure=True` over HTTPS
9	**No `Retry-After` or `X-RateLimit-*` headers.**	Clients cannot implement intelligent backoff and will retry blindly.	Emit the standard header set — section 8.2
10	**No Redis failure handling.** A connection error propagates out of the middleware as a 500.	Redis becomes a single point of failure for the entire API surface.	Wrap in `try/except` and choose fail-open or fail-closed explicitly

> **Priority order.** Items 1 and 3 are correctness and stability bugs and should be fixed first. Item 2 determines whether the limiter has any security value at all. Items 4 and 5 govern behaviour under load and become urgent the moment traffic is concurrent.

8. Future Improvements

8.1 Atomicity via Lua

Redis executes Lua scripts atomically, collapsing the check-and-update sequence into a single indivisible operation and eliminating the race described in limitation 4:

```lua
-- sliding_window.lua
local now, window, limit = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
redis.call('ZREMRANGEBYSCORE', KEYS[1], 0, now - window)
local count = redis.call('ZCARD', KEYS[1])
if count < limit then
  redis.call('ZADD', KEYS[1], now, ARGV[4])
  redis.call('EXPIRE', KEYS[1], window * 2)
  return 1
end
return 0
```

This also reduces three network round trips to one — a meaningful latency saving at scale, and the single most valuable change in this list.

8.2 Standard Rate Limit Headers

Emit these on every response, not only on rejections, so clients can self-regulate before they are throttled:

```
X-RateLimit-Limit: 5
X-RateLimit-Remaining: 2
X-RateLimit-Reset: 1721822410
Retry-After: 7
```

8.3 Additional Algorithms

- **Leaky Bucket** — enforces a strictly constant outflow rate; ideal in front of downstream systems with fixed throughput.
- **Sliding Window Counter** — a weighted hybrid of fixed and sliding windows that approximates sliding accuracy at fixed-window memory cost.
- **Concurrency limiting** — caps simultaneous in-flight requests rather than their rate; the right control for expensive long-running endpoints.

8.4 Distributed Concerns

- **Redis Cluster or Sentinel** for high availability; hash-tag keys so a single identity's state lands on one slot.
- **A local cache in front of Redis** — short-lived in-process counters flushed periodically, trading a little accuracy for a large latency reduction.
- **An explicit fail-open versus fail-closed policy** — decide what happens when Redis is unreachable and make it configurable per route. Health endpoints should fail open; payment endpoints should not.

8.5 Tiered and Route-Scoped Limits

- Different quotas per plan (`free` / `pro` / `enterprise`), resolved from the authenticated principal.
- Per-endpoint limits — `/login` should be far stricter than `/health` .
- A key format that carries all three dimensions: `ratelimit:{algorithm}:{route}:{identity}` .

8.6 Observability

SIGNAL	WHAT TO CAPTURE
Metrics	Counters for allowed versus denied requests, a histogram of Redis call latency, and a gauge of active limiter keys, exported to Prometheus
Structured logs	On every rejection: identity, route, algorithm, current usage, configured limit
Alerts	A spike in rejection rate (abuse, or a misconfigured limit) and any regression in Redis latency
Dashboards	Top consumers by volume — valuable for both capacity planning and abuse detection

> **End of document.** Rate Limiting Middleware — Technical Design & Implementation Reference, version 1.0.